INTER.act system documentation

Start here

INTER.act is a browser simulation of a production line. Robotic arms move objects from intake to output, and the whole setup is driven by one config file: `pipeline.yaml`.

If you only read one section, read this:

1. `pipeline.yaml` defines arms, dependencies, health policy, and optional frontend-only assets.
2. The backend (`run.py`) is state + API + Socket.IO broadcasting.
3. The pipeline logic runs in a separate demo-runner service (`scripts/demo_runner_server.py` + `scripts/run_demo.py`).
4. The frontend (`app/static/js/main.js`) renders from `/config` and live events.

Who this doc is for

- If you want to run or operate the system, jump to "Run and operate".
- If you want to understand architecture and data flow, read in order.
- If you need every route name, use "API surface".

This document keeps the main path short on purpose. Deep rendering internals and tuning constants belong in dedicated technical notes, not in the primary system overview.

System at a glance

```
Browser UI
  -> POST /demo/start
Backend API (:5000)
  -> proxy call to demo-runner (:6000)
Demo-runner
  -> executes pipeline steps via backend HTTP endpoints
Backend
  -> emits Socket.IO events
Browser
  -> animates arms/objects in real time
```

Runtime services:

- `backend` (`run.py`): Flask app, routes, in-memory state, Socket.IO broadcast, health monitor thread.
- `demo-runner` (`scripts/demo_runner_server.py`): accepts start/stop/status, runs pipeline in background.
- `arm-N` (`scripts/arm_server.py`): one microservice per arm for `/health`, `/move`, `/sequence`.
- `frontend`: static app + canvas renderer.

Single source of truth config

`app/models/pipeline_config.py` is the canonical configuration layer.

It owns:

- arm definitions and dependency graph
- scene config returned by `/config`
- valid object states (`intake`, `carried`, `handoff_1..N-1`, `output`)
- failure policy lookup
- health monitor settings

Other modules should consume this module instead of recomputing config-derived values.

Config shape

Core keys used by the system:

- `arms`
- `pipeline.mode`
- `failure_policy`
- `health`
- `assets` (frontend-only non-arm assets)

Notes:

- `pipeline.mode` is actively used by `scripts/run_demo.py`.
- `pipeline.products` and `pipeline.speed` may exist in YAML, but UI-triggered demo runs currently send fixed values from `app/static/js/ui/controls.js` (`products: 3`, `speed: "normal"`).

Pipeline execution model

`scripts/run_demo.py` drives arm actions and object movement using transfer buffers.

Supported modes:

- `sequential`: linear chain based on arm order.
- `parallel`: independent arms.
- `dependency`: dependency-driven order using `depends_on`.

Important implementation caveat:

- Dependency mode supports dependency graph validation and ordering, but runtime wiring is still effectively single-input/single-output per arm in key paths. Treat advanced branching DAG behavior as partial, not fully general.

Failure and recovery behavior

There are two failure paths and they behave differently:

1. Container-reported failure (`POST /arms/<id>/health` with `dead`)
    - marks the failed arm as dead
    - does not immediately hard-stop all downstream arms
    - downstream drain/stop transitions happen through pipeline flow
2. Explicit stop (`POST /arms/<id>/stop`)
    - cascades stop to downstream dependents immediately

Health monitoring:

- Backend health monitor thread (`app/services/health_monitor.py`) polls arm `/health`.
- Missed checks move arms from `alive` to `unhealthy`, then to `dead`.
- Recovery only auto-applies to `dead` and `stopped` when upstream dependencies are healthy.
- `draining` is intentionally not auto-recovered by the monitor.

Frontend model

Frontend entry is `app/static/js/main.js`.

Main responsibilities:

- fetch `/config`
- keep local render state (`core/state.js`)
- consume Socket.IO updates (`core/socket.js`)
- run frame loop (`core/tick.js` + `core/render.js`)

Asset types:

- `arm`: backend-driven
- `vehicle` / `conveyor`: frontend-driven simulation layer from `assets` config

The backend remains authoritative for arm and object lifecycle states. The frontend can add local transit visuals for non-arm assets.

API surface

Page and health:

- `GET /`
- `GET /health`
- `GET /health/ollama`

Arm and object state:

- `POST /set_angle`
- `GET /state`
- `GET /state/<id>`
- `POST /object_event`

- `GET /objects`
- `POST /objects/reset`
- `POST /log`

Config and pipeline lifecycle:

- `GET /config`
- `POST /reload_config` (reloads YAML in memory only)
- `POST /restart_pipeline` (reinitializes runtime state only)
- `GET /pipeline/status`

Health and control:

- `GET /arms/health`
- `POST /arms/<id>/health`
- `POST /arms/<id>/stop`
- `GET /arms/<id>/allowed`

Demo proxy routes:

- `POST /demo/start`
- `GET /demo/status`
- `POST /demo/stop`

Important caveat:

- `/demo/stop` reflects demo-runner status but is not a guaranteed hard interrupt of all in-flight worker actions in the current implementation.

Run and operate

Docker

```
docker compose up --build
# open http://localhost:5000
```

Local

```
pip install -r requirements.txt
python run.py
python scripts/run_demo.py --speed normal --products 3
```

Update config safely

```
# 1) reload file into config cache
curl -X POST http://localhost:5000/reload_config
```

```
# 2) apply to runtime state
curl -X POST http://localhost:5000/restart_pipeline
```

`/reload_config` and `/restart_pipeline` are intentionally separate operations.

Scale arm containers after config changes

```
python scripts/generate_compose.py
docker compose up --build
```

Test

```
pytest tests/ -v
```

Known constraints

- Dependency-mode runtime behavior is not fully generalized for complex branching DAG execution paths.
- Browser reconnect state is assembled from multiple endpoints, so snapshots are not fully atomic.
- Socket updates are frequent and can be chatty under heavy movement.

Related docs

- `docs/api_reference.md`
- `docs/failure_model.md`
- `docs/frontend_assets.md`
- `docs/troubleshooting.md`
- `docs/docker_stack.md`
- `docs/backend_runtime.md`
- `docs/config_reference.md`
- `docs/development_workflow.md`

Use this file as the architecture map, then jump into the focused docs as needed.